OFFICIAL ARCHITECTURE & ASSET SPECIFICATION MANUAL

Binaare Fine Art Gallery

Complete Architecture, Page Directory, Image Asset Specifications, Mandatory Node.js Server Runtime & Future Engineering Roadmap

This document serves as the comprehensive engineering and production specification for the Binaare Art Gallery digital platform. It details full technology stack dependencies, complete page architecture, exact dimensional blueprints for artwork components (Selected Works and Product View), server environment requirements (including mandatory Node.js execution), and implementation plans for payment gateways, CMS administration, and email automation.

PLATFORM & FRAMEWORK
Next.js 15.1.0 • React 19 • App Router

MANDATORY RUNTIME
Node.js >= 22.18.0 LTS

DOCUMENT CLASSIFICATION
Technical Engineering & Operations Manual

RELEASE VERSION
Version 2.0 (Production Master)

PRIMARY ARTIST / CLIENT
Binari Gamage (Binaare Art)

IMAGE & DATABASE ENGINE
Node Native SQLite + Sharp C++ WebP Pipeline

Executive Summary & Table of Contents

OVERVIEW

The **Binaare Art Gallery** website is an online fine art gallery designed to showcase and sell paintings by artist Binari Gamage. The platform combines visual aesthetics—such as smooth 60fps scrolling, responsive layouts, and artwork zoom—with modern software engineering.

DOCUMENT PURPOSE & OPERATIONAL AUDIENCE

This manual provides engineers, system administrators, content managers, and designers with technical standards for image replacement, server deployment, database persistence, and future e-commerce integration.

SECTION	MODULE TITLE	CORE TECHNICAL FOCUS
Section 01	**Full Technology Stack Breakdown**	Next.js 15, React 19, Tailwind v4, Framer Motion, GSAP, Sharp, and SQLite.
Section 02	**Website Structure & All Page Details**	14 distinct routes, layout architectures, data flows, and interactive features.
Section 03	**"Selected Works" Section: Image & Card Sizing**	Card dimensions (width & height), aspect ratio (4:4.5), and replacement guide.
Section 04	**"Product View" Section: Image & Frame Sizing**	Showcase frame dimensions, high-res zoom (1600–2400px), and thumbnail gallery.
Section 05	**Master Asset Dimension Reference Matrix**	Comparative cross-platform sizing table covering every website component.
Section 06	**Future Development: Admin CMS & Payment Gateway**	Enterprise Admin CMS, Yamnet/PayGate Payment Gateway architecture & flows.
Section 07	**Future Development: Transactional Email Service**	Resend/SendGrid/Nodemailer email dispatch, triggers, and receipts.
Section 08	**Server Compatibility: Mandatory Node.js (Part 1)**	**Mandatory Node.js runtime**, VPS/PM2/Nginx architecture, and Docker.
Section 09	**Server Compatibility: cPanel, Env Variables & Sign-Off**	cPanel CloudLinux setup, Environment Variables, and Engineering Sign-Off.

Technology Stack & Dependencies

SECTION 01

The web platform is built on an enterprise modern JavaScript/TypeScript stack utilizing Next.js 15 App Router architecture, React 19, and Tailwind CSS v4. The platform leverages both server-rendered and interactive client components to guarantee ultra-fast initial page delivery and zero layout shifts.

1. Core Framework & Infrastructure

TECHNOLOGY	VERSION	ROLE & ARCHITECTURAL IMPLEMENTATION
Next.js	15.1.0	Full-stack React framework utilizing App Router architecture, React Server Components (RSC), and dynamic Route Handlers (`/api` endpoints).
Node.js Runtime	>= 22.18.0 LTS	**Mandatory execution engine.** Provides native built-in `node:sqlite` database, modern ES modules, and crypto APIs.
React & React-DOM	19.0.0	Latest React core with Actions, Transitions, and enhanced Server Component hydration.
TypeScript	5.X	Strict compile-time type-safety for models, APIs, and component props.
Tailwind CSS	4.0.0	Utility-first styling engine running via `@tailwindcss/postcss` for sub-millisecond compilation.
Database Engine	SQLITE / MYSQL2	Native Node SQLite (`data/gallery.sqlite`) for zero-latency local relational storage; `mysql2` driver included for cloud scale.
Image Processor	SHARP 0.34.5	C++ native image transformer. Auto-rotates EXIF orientations, resizes up to 2400px, and converts uploads to high-compression WebP.

2. Animation, Motion & UX Libraries

LIBRARY	VERSION	PURPOSE & EXPERIENCE IMPACT
Framer Motion	12.38.0	Orchestrates scroll reveals, card hover transforms, stagger animations, and modal state transitions.
GSAP	3.15.0	GreenSock Animation Platform powering hero parallax effects, luxury timing curves, and timeline triggers.
Lenis	1.3.23	Smooth inertial scrolling engine eliminating choppy browser scrolling for a fluid art gallery atmosphere.
Lucide React	1.14.0	Consistent, lightweight SVG iconography across navigation, filters, product specifications, and cart drawers.
Zod	4.4.3	Schema validation for admin forms, artwork uploads, blog payloads, and checkout fields.
Playwright	1.63.0	End-to-end integration and backend validation testing on automated headless Chrome instances.

Website Structure & Page Details

SECTION 02

The Binaare Art Gallery site map encompasses 14 core route endpoints, structured into public art presentation zones, an e-commerce acquisition flow, an editorial art journal, and a protected administrative content studio.

ROUTE PATH	PAGE TITLE	KEY FEATURES & LAYOUT COMPONENTS	DATA SOURCE
`/`	**Homepage**	Parallax ambient hero video/fallback, Intro statement, **Selected Works** 4-column card grid, Artist preview, Philosophy quote, Newsletter opt-in.	HYBRID
`/shop`	**Fine Art Shop**	Dynamic medium pills (Oil, Watercolor, Mixed Media, Texture, Acrylic), Live search, Sorting engine, Grid/List switcher, Favorites wishlist, Add to Cart.	LIVE DB
`/shop/[slug]`	**Artwork Detail View**	2-column luxury layout: Sticky left artwork showcase with hover zoom & badges, Right column narrative, Medium, Dimensions, Materials, Direct checkout button.	LIVE DB
`/gallery`	**Exhibition Gallery**	3-column masonry grid layout with variable aspect ratios, zoom lightbox, medium filters, curatorial blurbs.	CONTENT
`/about`	**About the Artist**	Artist biography (Binari Gamage), Studio photographs, Curatorial philosophy, Exhibition timeline, Press & honors.	CONTENT
`/artist`	**Artist Profile**	In-depth aesthetic journey, inspiration, technique deep dive, materials exploration, studio portraits.	CONTENT
`/exhibitions`	**Exhibitions**	Curated past and upcoming gallery exhibitions, dates, venues, featured painting collections.	CONTENT
`/blog`	**Editorial Journal**	Curated essays, behind-the-canvas reflections, artistic techniques, publication date stamps, featured photography.	LIVE DB
`/blog/[slug]`	**Article Detail**	Long-form editorial reading experience, preserved linebreaks, embedded photo gallery, author bio.	LIVE DB
`/cart`	**Shopping Cart**	Interactive acquisition drawer & full page: Item list, quantity adjustments, removal, subtotal calculation, checkout link.	CLIENT STATE
`/checkout`	**Acquisition Checkout**	Shipping address capture, client details, order summary review, payment initiation interface.	CART CONTEXT
`/contact`	**Private Inquiries**	Artwork acquisition form, private studio viewing requests, bespoke commission inquiries, studio contact info.	FORM API
`/pages/[slug]`	**Dynamic Pages**	Custom landing pages created dynamically through the admin studio, including custom sections and images.	LIVE DB
`/admin`	**Content Studio**	Authenticated back-office CMS: Artwork manager, Blog publisher, Page section customizer, Media library with Sharp uploads.	SECURE AUTH

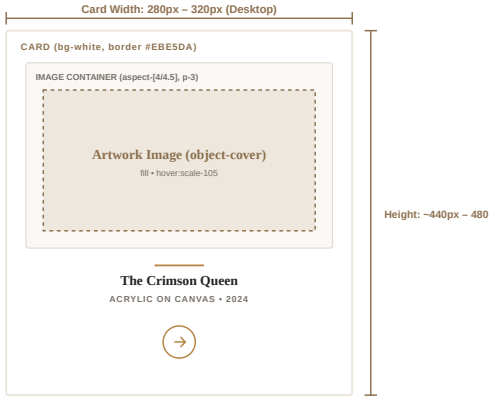
"Selected Works" Section: Image & Card Sizing

SECTION 03

The **Selected Works** section is positioned on the homepage (`/`, defined in `src/app/page.tsx` lines 301–370). It features 4 high-priority original paintings in an editorial card layout.

1. Visual Dimensional Blueprint (Width & Height Specifications)

DIMENSIONAL BLUEPRINT: SELECTED WORKS ARTWORK CARD



TARGET ASPECT RATIO

4 : 4.5 (Approx 8 : 9)

Height is 1.125× of Width. Preserves vertical elegance.

RECOMMENDED UPLOAD SIZE

1200 × 1350 px (2× Retina)

Standard minimum acceptable: 800 × 900 px.

RESPONSIVE BREAKPOINTS

Desktop: 4 Cols • Tablet: 2 Cols • Mobile: 1 Col

Card scales fluidly from 280px up to 420px max on mobile.

FORMAT & SHARP PIPELINE

WebP (Primary), Progressive JPG, PNG

Max 10 MB. Auto-compressed & normalized by Sharp.

2. Exact Specifications & Step-by-Step Replacement

CARD DISPLAY DIMENSIONS

Width: 280–320px | Height: 440–480px

Desktop 4-column layout; scales to 100% width (up to 420px) on mobile.

IMAGE BOX ASPECT RATIO

aspect-[4/4.5] (8 : 9 Ratio)

CSS rule: `aspect-[4/4.5] bg-[#FAF8F5] p-3`.

RECOMMENDED UPLOAD RESOLUTION

1200 px (Width) × 1350 px (Height)

High-DPI 2x density. Minimum acceptable: 800 × 900 px.

SUPPORTED FORMATS & MAX SIZE

WebP, Progressive JPG, PNG (Max 10 MB)

Converted automatically to optimized WebP by the Sharp engine.

HOW TO REPLACE SELECTED WORKS IMAGES

- **Method A (via Admin Studio at `/admin` — Recommended):** Log in to `/admin` → Click *Pages & Sections* → Select *Home Page* → Find *Featured Collection / Selected Works* → Upload or select new painting from Media Library → Click *Save Changes*. Live immediately.
- **Method B (via Codebase Asset File):** Save optimized image to `public/artworks/[artwork-slug].webp`. Update reference in `src/data/products.ts` or update the SQLite database seeded in `data/gallery.sqlite`.

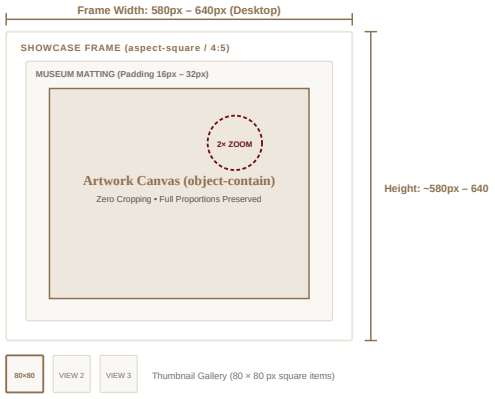
"Product View" Section: Image & Frame Sizing

SECTION 04

The **Product View** (Artwork Detail Page, located at `/shop/[slug]`, rendered by `ProductDetailView.tsx` and `ProductImages.tsx`) provides an art acquisition experience. The left column displays the artwork in a presentation frame with hover zoom, while the right column details the narrative, dimensions, and purchase controls.

1. Visual Dimensional Blueprint (Width & Height Specifications)

DIMENSIONAL BLUEPRINT: PRODUCT DETAIL SHOWCASE FRAME & ZOOM



TARGET ASPECT RATIO

1 : 1 or 4 : 5 (Adaptive)

Square showcase box or tall canvas portrait format.

RECOMMENDED MASTER IMAGE SIZE

1600 × 1600 up to 2400 × 2400 px

Provides crystal clarity during interactive 2× magnifier zoom.

PRESENTATION PHYSICS

object-contain with Museum Matting

Original canvas proportions are never cropped or stretched.

THUMBNAIL GALLERY SPECS

80 × 80 px (1:1 Square) • 2px Gold Active Border

Supports: Cover, Heavy Texture Angle, Room Preview, Signature.

2. Exact Frame Specifications & Image Replacement Guide

SHOWCASE FRAME DIMENSIONS

Width: 580–640px | Height: 580–640px

Sticky left column (`w-full lg:w-[48%]`); 100% width on tablet/mobile.

IMAGE RENDERING MODE

object-contain with Museum Matting

Paintings retain their exact organic canvas aspect ratio (never stretched or cropped).

RECOMMENDED MASTER IMAGE RESOLUTION

1600 × 1600 px to 2400 × 2400 px

Required to maintain high clarity during interactive hover zoom (`scale(2)`).

THUMBNAIL GALLERY DIMENSIONS

80 px × 80 px (Square 1:1)

Container: `80px x 80px`, `object-fit: cover`.

HOW TO ADD & REPLACE PRODUCT IMAGES

- **Primary Photo (Cover View):** Uploaded as the first image in the product's image array. Displays as the default showcase painting and the catalog thumbnail.
- **Multiple High-Res Gallery Views:** Administrators can upload up to 5 additional photographs per artwork (e.g. angle view, heavy texture close-up, room preview, artist signature).
- **Auto-Optimization:** The backend Sharp pipeline accepts JPG, PNG, WebP, or AVIF up to 10 MB and 40 megapixels, automatically compressing it to WebP at a max dimension of 2400px.

Master Asset Dimension Reference Matrix

SECTION 05

To ensure consistent visual quality across the entire gallery website, all media assets must adhere to the following target dimensions, aspect ratios, and format standards.

SECTION / COMPONENT	CARD / FRAME SIZE	ASPECT RATIO	RECOMMENDED IMAGE SIZE	OBJECT FIT	FORMAT
Selected Works (Home) `src/app/page.tsx`	280 × 460 px (Desktop col)	`4 : 4.5` (8:9 ratio)	**1200 × 1350 px** (Min: 800×900px)	`object-cover`	WebP / JPG
Product View (Detail) `ProductDetailView.tsx`	580 × 580 px (Desktop 48% col)	`1 : 1` or `4 : 5` (Square / Port)	**1600 × 1600 px** (Up to 2400×2400px)	`object-contain`	WebP (High-Res)
Product Thumbnails `ProductImages.tsx`	80 × 80 px (Fixed square)	**1 : 1** (Square)	**160 × 160 px** (Auto-scaled)	`object-cover`	WebP
Shop Catalog Card `src/app/shop/page.tsx`	280 × 420 px (4-col grid)	`4 : 3` (Landscape)	**1200 × 900 px** (Min: 800×600px)	`object-cover`	WebP / JPG
Masonry Gallery Grid `src/app/gallery/page.tsx`	Dynamic col (3-col masonry)	**VARIABLE** (Original ratio)	**1440 × 1440 px** (Adaptive height)	`aspect-auto`	WebP
Hero Ambient Visual `SeamlessHeroVideo.tsx`	100vw × 100vh (Full viewport)	**16 : 9** (Widescreen)	**1920 × 1080 px** (Video MP4 & Poster)	`object-cover`	MP4 / WebP
Artist Portrait / Bio `src/app/about/page.tsx`	420 × 630 px (Editorial col)	`2 : 3` (Vertical port)	**1600 × 2400 px** (Studio quality)	`object-cover`	WebP / JPG
Blog Editorial Covers `src/app/blog/page.tsx`	360 × 240 px (Article card)	**16 : 10** (Editorial)	**1280 × 800 px** (Min: 960×600px)	`object-cover`	WebP / JPG

Future Development: Admin CMS & Payment Gateway

SECTION 06

To transform the current platform into a full-scale commercial art enterprise, three critical modules have been architected: an Enterprise Admin Panel, a secure Payment Gateway (Yamnet / International / Local), and an automated Transactional Email Service.

1. Enterprise Admin Panel CMS Evolution

- **Multi-User Role-Based Access Control (RBAC):** Granular permissions separating Super Admin (financials, payments), Curator (artworks, exhibitions, pricing), and Editor (blog, stories).
- **Artwork Inventory & Provenance Tracking:** Management of status states: *Available*, *Reserved*, *Acquired / Private Collection*, or *On Exhibition*. Automatic certificate issuance tracking.
- **Automated Certificate of Authenticity (CoA) Generator:** Server-side PDF generation producing signed, numbered, digital certificates containing high-res artwork thumbnails and artist signature upon purchase.
- **Customer & Inquiry CRM:** Dedicated inbox dashboard for VIP viewing requests, private commission inquiries, and newsletter subscriber exports.

2. Payment Gateway Integration (Yamnet / Gateway Engine)

The cart currently functions via client-side state. The next release connects the checkout flow to a secure payment gateway (Yamnet, PayGate, PayHere, or Stripe) using server-side Next.js route handlers.

ARCHITECTURE: PAYMENT GATEWAY TRANSACTION FLOW

```
// Step 1: Customer submits checkout form on /checkout
[Client Browser] → POST /api/checkout/create-order
  • Validates items, prices, shipping address against backend SQLite/MySQL DB
  • Generates unique order ID (e.g. BAG-2026-0849) with status: "PENDING_PAYMENT"

// Step 2: Server initiates gateway session with Yamnet / PayGate API
[Next.js Server] → POST https://gateway.yamnet.com/v1/checkout/session
  • Headers: Authorization Bearer <SECRET_API_KEY>
  • Payload: { order_id, amount, currency: "USD", return_url, webhook_url }
  ← Returns: { checkout_token, redirect_url }

// Step 3: Redirect customer to secure hosted payment page
[Client Browser] → Redirects to Gateway Hosted Payment Page (PCI-DSS Level 1)

// Step 4: Webhook Listener validates payment asynchronously
[Yamnet Gateway] → POST /api/webhooks/payment (To Binaare Server)
  • Server verifies cryptographic HMAC-SHA256 signature
  • Updates order status to "PAID"
  • Decrements artwork inventory (marks painting as Sold)
  • Dispatches customer receipt & admin order alert via Transactional Email
```

Future Development: Transactional Email Service

SECTION 07

An integrated email dispatch pipeline (utilizing **Resend**, **SendGrid**, or **Nodemailer** with SMTP) handles critical communications:

EVENT TRIGGER	RECIPIENT & TEMPLATE	PAYLOAD CONTENT & ATTACHMENTS
Order Completed (Paid)	Customer • Luxury Receipt Template	Order summary, shipping address, tax breakdown, downloadable Certificate of Authenticity PDF.
New Acquisition Alert	Gallery Director / Artist • Alert	Collector name, phone, email, purchased artwork title, total amount, shipping tracking prompt.
Private Viewing Request	Artist (Binari Gamage) • VIP Lead	Prospective collector details, desired artwork, preferred date/time, custom message.
Newsletter Subscription	Subscriber • Welcome Letter	Curated welcome note from the artist, invitation to upcoming private gallery viewings.

Email Dispatch Integration Code Example

```
// src/lib/email.ts — Transactional Email Service with Resend / Nodemailer
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

export async function sendOrderConfirmationEmail(order: {
  id: string;
  customerName: string;
  customerEmail: string;
  artworkTitle: string;
  amount: number;
  pdfCertificateUrl: string;
}) {
  return await resend.emails.send({
    from: 'Binaare Fine Art <acquisitions@binaareart.com>',
    to: order.customerEmail,
    subject: `Acquisition Confirmed: ${order.artworkTitle} — Binaare Art Gallery`,
    html: `<div style="font-family: Georgia, serif; color: #24211F;">
      <h1 style="color: #8C704F;">Thank you for your acquisition, ${order.customerName}.</h1>
      <p>We have received your payment for <strong>${order.artworkTitle}</strong>.</p>
      <p>Your hand-signed Certificate of Authenticity is attached.</p>
    </div>`,
  });
}
```

Server Compatibility: Mandatory Node.js Requirement

SECTION 08

> **⚠ CRITICAL SYSTEM REQUIREMENT: SERVER MUST RUN NODE.JS**
>
> **The server MUST run Node.js (Version >= 22.18.0 LTS).** Standard cheap shared web hosting running Apache/PHP alone **CANNOT** run this dynamic platform. The Next.js 15 App Router backend, the native SQLite storage (`node:sqlite`), the authenticated Admin Studio, dynamic Sharp image processing, and payment webhooks require a live Node.js application process.

1. Technical Root Cause: Why Node.js is Required

- **Native Node.js SQLite API (`node:sqlite`):** The content storage engine directly binds to Node's built-in SQLite module introduced in Node 22.18.0. Traditional PHP environments do not execute this.
- **Next.js Dynamic Route Handlers:** APIs at `/api/admin`, `/media/[filename]`, and future payment webhooks are compiled as Node.js HTTP request handlers.
- **C++ Sharp Image Pipeline:** Image uploads are transcoded on-the-fly using Sharp's compiled native Node.js binaries.
- **Encrypted Session Cookies & Auth:** Admin sessions, password verification with salted SHA-256 hashes, and CSRF protection execute inside the Node.js server lifecycle.

2. Minimum Server Hardware & OS Specifications

OPERATING SYSTEM	NODE.JS RUNTIME
Ubuntu 22.04 / 24.04 LTS or Debian 12 Linux x86_64 or ARM64 (AWS Graviton / Mac Silicon supported).	**Node.js v22.18.0 LTS or newer** Must include active npm / corepack tools.
MEMORY (RAM)	**STORAGE & PERMISSIONS**
Minimum 2 GB RAM (Recommended: 4 GB) Provides sufficient memory headroom during Sharp image compression.	**Persistent NVMe SSD with Write Access** Writable permissions required on `./data/` and `./data/uploads/`.

3. Recommended Production Deployment Architectures

OPTION A: LINUX VPS / DEDICATED SERVER (DIGITALOCEAN, AWS EC2, HETZNER, LINODE)

```
# 1. Install Node.js 22 LTS via NodeSource
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs build-essential

# 2. Clone repository & install production dependencies
cd /var/www/binaare-art-gallery
npm ci && npm run admin:setup && npm run build

# 3. Start persistent server with PM2 Process Manager
npm install -g pm2
pm2 start npm --name "binaare-gallery" -- start
pm2 save && pm2 startup
```

OPTION B: DOCKER CONTAINER DEPLOYMENT

```
FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production PORT=3000
COPY .next/standalone ./
COPY .next/static ./.next/static
COPY public ./public
VOLUME ["/app/data"]
EXPOSE 3000
CMD ["node", "server.js"]
```

cPanel Setup, Environment Variables & Sign-Off

SECTION 09

1. cPanel Hosting (CloudLinux Node.js Application Manager)

If hosted on cPanel, the hosting plan **MUST** have *"Setup Node.js App"* (CloudLinux / Phusion Passenger) enabled:

- Open cPanel → Click **Setup Node.js App**.
- Select **Node.js Version: 22.x** (or highest available ≥ 22.18).
- Set **Application Mode** to *Production*.
- Set **Application Root** to `/home/username/binaare`.
- Set **Application Startup File** to `server.js` or `node_modules/next/dist/bin/next` with argument `start`.
- Add Environment Variables: `BINAARE_DATA_DIR`, `APP_ORIGIN`, `NODE_ENV=production`.

2. Environment Variables Checklist

VARIABLE NAME	EXAMPLE VALUE	FUNCTIONAL PURPOSE
`NODE_ENV`	`production`	Enables high-performance minified server execution and caching.
`PORT`	`3000`	HTTP port listener for the Node.js server process.
`BINAARE_DATA_DIR`	`/var/www/binaare/data`	Absolute directory for SQLite database and uploaded media files.
`APP_ORIGIN`	`https://binaareart.com`	Origin verification header preventing CSRF attacks on admin mutations.
`ADMIN_USERNAME`	`admin`	Configured username during automated setup.
`ADMIN_PASSWORD`	`********`	Secure master password (minimum 12 characters).

ENGINEERING SIGN-OFF & ARCHITECTURAL APPROVAL

Document ID: BAG-TECH-2026-FINAL

LEAD SOFTWARE ARCHITECT	PLATFORM RUNTIME STATUS	DOCUMENT REVISION
Antigravity Systems Engineering	**Node.js 22.18+ Verified Active**	**2.0.0 (Production Master)**